

PratoFácil.

API REST para Gestão de Pedidos Locais

POST /api/pedidos → 201 CREATED

// O PROBLEMA

Caderno e WhatsApp **não** **escalam**

Pequenos empreendedores de comida
ainda anotam e controlam tudo à mão.

01

Pedidos perdidos

Informação some entre conversas e
anotações.

02

Erros no atendimento

Itens trocados, contas erradas,
retrabalho.

03

Sem visão do negócio

Difícil acompanhar vendas e crescer.

// A SOLUÇÃO

Um só lugar, do **cardápio** à **entrega**

Um marketplace onde cada loja tem o seu cardápio, e o cliente escolhe onde pedir.

RF01

Listar o cardápio

Pratos por categoria, abertos a qualquer cliente.

RF02

Fazer o pedido

O cliente pede por uma requisição HTTP.

RF03

Atualizar o status

O lojista acompanha até a entrega.

// POR QUE É UM SISTEMA DISTRIBUÍDO

Três peças, uma conversa por HTTP



Cliente • Servidor • Banco

Componentes separados que se comunicam pela rede.



Transparência

O cliente usa o serviço sem saber onde o servidor e o banco estão hospedados.



Middleware

O Spring Boot intermedeia requisições, regras e dados.

// ARQUITETURA

Como as peças se conectam



Padrão **MVC + camada de serviço** · a mesma lógica exposta em **REST (JSON)** e **Web (Thymeleaf)** · ramo síncrono → **Gateway de pagamento**.

// REST SOBRE HTTP

Verbos, JSON e status codes

GET	/api/pratos	200 OK
POST	/api/pedidos	201 CREATED
PUT	/api/pedidos/1/status	200 OK
DELETE	/api/pratos/9	409 CONFLICT

Erros de domínio em [JSON padronizado](#) (sem stack trace) • documentação viva em [Swagger](#) / [OpenAPI](#).

// INTEGRAÇÃO COM WEBSERVICE EXTERNO

Pagamento: dois sistemas conversando

SÍNCRONO

App → Asaas

O servidor consome a API do Asaas (RestClient): cria a cobrança **PIX** com QR Code ou **captura o cartão na hora**.

ASSÍNCRONO

Asaas → App

Um **webhook** em `/webhooks/asaas` confirma o pagamento. Orientado a eventos — sem ficar perguntando (polling).

- Nesta apresentação, o pagamento é **simulado** — sem cobrança real.

// SERVIÇOS EM NUVEM

Mesmo código, qualquer lugar

DEV

H2 em arquivo

Banco SQL leve e **embutido na aplicação** (gravado em arquivo) — ideal para desenvolver e testar.

PROD

PostgreSQL no Render

Banco em nuvem — só **troca o perfil**, o código não muda.

● App em Docker, no ar: <https://pratofacil.onrender.com>

Transparência de localização: muda o endereço do banco, não a aplicação.

// SEGURANÇA & QUALIDADE

Protegido e testado

SEGURANÇA

Papéis & isolamento

ADMIN e **CLIENTE** · senhas com **BCrypt** · cada um enxerga só os próprios dados.

TESTES

7 automatizados ✓

Integração com **MockMvc** + testes manuais (Postman).
Status 200 · 201 · 204 · 400 · 401 · 404 · 409 validados.

Cliente nunca informa preços: o **valor total** é calculado no servidor.

// O GRUPO

Integrantes

01 Antônio Henrique Torres Silva

04 Caio Rafael da Encarnação Freitas

07 Felipe Barbosa da Silva

10 Matheus de Paula Vieira

13 Thayane Moura Silva

02 Bianca Rodrigues dos Santos

05 Carlos Daniel Batista

08 Kleyber Daniel Maia Barreto Noronha

11 Natan Vigil de Azambuja

03 Breno Gabriel da Silva

06 Enzo Marinho Machado Vieira

09 Márcio dos Anjos

12 Pedro Henrique Ferreira da Silva

// CONCLUSÃO

Sistemas Distribuídos, na prática

cliente-servidor

REST / HTTP

middleware

transparência

nuvem

webhook

Evolução: notificações • paginação • pagamento assíncrono • app mobile.

Obrigado!

<https://pratofacil.onrender.com>



Aponte a câmera
e acesse o app no ar